

Projektowanie Efektywnych Algorytmów

Problem Komiwożacza (ATSP)

Marcel Cieśliński

Nr albumu: 280871

25 Marca 2026

Kurs	Projektowanie Efektywnych Algorytmów - Projekt
Temat projektu	Asymetryczny Problem Komiwożacza: Implementacja i analiza algorytmu Branch and Bound - Zadanie 2
Prowadzący	dr inż. Jarosław Mierzwa
Termin oddania	26 marca 2026 r.
Autor	Marcel Cieśliński

1 Wstęp teoretyczny

1.1 Opis problemu

Asymetryczny Problem Komiwojażera (ATSP) polega na znalezieniu najtańszego cyklu Hamiltona w grafie. "Asymetrycznym" oznacza, że koszt przejazdu z miasta A do B może być inny niż z B do A.

1.2 Złożoność obliczeniowa

Problem ATSP należy do klasy problemów NP-trudnych. Rozwiązanie go metodą Brute Force (przegląd zupełny) posiada złożoność czasową $O(N!)$. Metoda Podziału i Ograniczeń (Branch and Bound) nie zmienia pesymistycznej złożoności – w najgorszym przypadku algorytm nadal musiałby sprawdzić wszystkie permutacje. Jednak w praktyce średnia złożoność jest niższa. Dzięki mechanizmowi odcinania gałęzi, B&B pozwala na rozwiązywanie znacznie większych instancji problemu w akceptowalnym czasie.

2 Przykład praktyczny

Rozpatrzono działanie algorytmu dla instancji problemu o rozmiarze $N = 4$. Koszt trasy zdefiniowano losowo w zakresie 10 – 99, a jedną z wartości (koszt z miasta 0 do 3) zastąpiono dwiema ostatnimi cyframi (71). Zastosowano strategię przeszukiwania Best-First Search (Kopiec Min).

Macierz początkowa (kosztów)

$$M = \begin{bmatrix} \infty & 15 & 20 & 71 \\ 12 & \infty & 30 & 18 \\ 45 & 10 & \infty & 25 \\ 22 & 35 & 14 & \infty \end{bmatrix}$$

Krok 1: Redukcja macierzy

Algorytm wyznacza minimalny koszt opuszczenia i wjazdu do każdego miasta.

- **Redukcja wierszy:**

- Wiersz 0: $\min = 15$. Po odjęciu: $[\infty, 0, 5, 56]$
- Wiersz 1: $\min = 12$. Po odjęciu: $[0, \infty, 18, 6]$
- Wiersz 2: $\min = 10$. Po odjęciu: $[35, 0, \infty, 15]$
- Wiersz 3: $\min = 14$. Po odjęciu: $[8, 21, 0, \infty]$

Suma redukcji wierszy: $15 + 12 + 10 + 14 = 51$

- **Redukcja kolumn:**

- Kolumny 0, 1 i 2 posiadają już zera.
- Kolumna 3: $\min = 6$. Odejmujemy 6 od pozostałych elementów.

Suma redukcji kolumn: 6

Dolne ograniczenie (Lower Bound) dla węzła startowego: $51 + 6 = 57$

Macierz po pełnej redukcji prezentuje się następująco:

$$M' = \begin{bmatrix} \infty & 0 & 5 & 50 \\ 0 & \infty & 18 & 0 \\ 35 & 0 & \infty & 9 \\ 8 & 21 & 0 & \infty \end{bmatrix}$$

Krok 2: Generowanie ścieżek z miasta 0 (Podział)

Z miasta początkowego generowane są ścieżki do pozostałych miast. Obliczmy koszt wygenerowania gałęzi dla trasy $0 \rightarrow 1$:

- Koszt przejścia pobrany ze zredukowanej macierzy M' wynosi: 0.
- Blokowanie powrotów: w macierzy ustawiamy wiersz 0 oraz kolumnę 1 na ∞ . Dodatkowo blokujemy natychmiastowy powrót ustawiając $M'[1][0] = \infty$.
- Wykonujemy ponowną redukcję tak przygotowanej macierzy wiersz po wierszu i kolumna po kolumnie.
- *Nowy Lower Bound = Stary Bound (57) + Koszt przejścia (0) + Koszt nowej redukcji.*

Wszystkie wygenerowane w ten sposób węzły trafiają do Kopca Min. W kolejnej iteracji algorytm nie analizuje węzłów sekwencyjnie, lecz wyciąga z kopca ten o najniższym szacowanym koszcie.

3 Opis implementacji algorytmu

- **Funkcja obliczająca ograniczenie (ReduceMatrix):** Funkcja składa się z dwóch głównych etapów. Pierwsza pętla iteruje po wszystkich wierszach macierzy, odnajdując w każdym najmniejszą wartość i odejmując ją od pozostałych elementów danego wiersza. Druga pętla analogicznie redukuje kolumny. Suma wyciągniętych wartości z obu etapów stanowi koszt redukcji.
- **Struktury Przeszukiwania (Kopiec vs. Kolejka):** W projekcie zaimplementowano własne struktury danych. Zwykła kolejka FIFO obsługuje wariant przeszukiwania wszerz (Breadth-First Search). Z kolei wariant Best-First Search opiera się na własnoręcznie napisanej strukturze Kopca Min (MinHeap), gdzie metody `heapifyUp` i `heapifyDown` dbają, by zawsze obsługiwać trasę o najwyższym priorytecie (najniższym `lowerBound`).
- **Zarządzanie pamięcią:** Aby zapobiec wyciekowi pamięci w trakcie głębokiej rekurencji drzewa, reprezentacja macierzy wykorzystuje obiekty `std::vector`. Umożliwia to elastyczne i bezpieczne przekazywanie stanu wewnątrz programu.

4 Plan eksperymentu

4.1 Środowisko sprzętowe i pomiar czasu

Pomiary zostały przeprowadzone za pomocą biblioteki `<chrono>` (zegar `high_resolution_clock`). Środowisko sprzętowe platformy mobilnej, na której przeprowadzono testy, zestawiono w Tabeli 1.

Komponent	Specyfikacja
Procesor (CPU)	AMD Ryzen 5 7520U (Mobile)
Pamięć operacyjna (RAM)	16 GB
Pamięć masowa	Dysk SSD
Środowisko programistyczne	Visual Studio (C++) / Tryb Release

Tabela 1: Specyfikacja sprzętowa środowiska testowego

4.2 Sposób generowania danych testowych

Macierze kosztów generowane są w sposób zautomatyzowany. Koszty przejścia między poszczególnymi miastami losowane są z równomiernym prawdopodobieństwem z przedziału wartości całkowitych $\langle 1, 100 \rangle$. Główna przekątna macierzy ($i = j$) wypełniana jest wartością -1 , co przez zaimplementowane algorytmy interpretowane jest jako nieskończoność (∞), blokując w ten sposób generowanie niepoprawnych tras prowadzących do tego samego miasta.

5 Wyniki eksperymentów i wnioski

5.1 Porównanie z algorytmem dokładnym (Brute-Force)

Aby uwidocznąć skuteczność metody Branch and Bound, w Tabeli 2 przypominano czasy działania algorytmu Przeglądu Zupełnego (zrealizowanego w pierwszym zadaniu projektowym).

Liczba miast (N)	Czas wykonania Brute-Force (ms)
10	3,16
11	33,79
12	400,74
13	5150,68
14	68863,80

Tabela 2: Czas wykonania algorytmu Brute-Force (dane z Projektu 1)

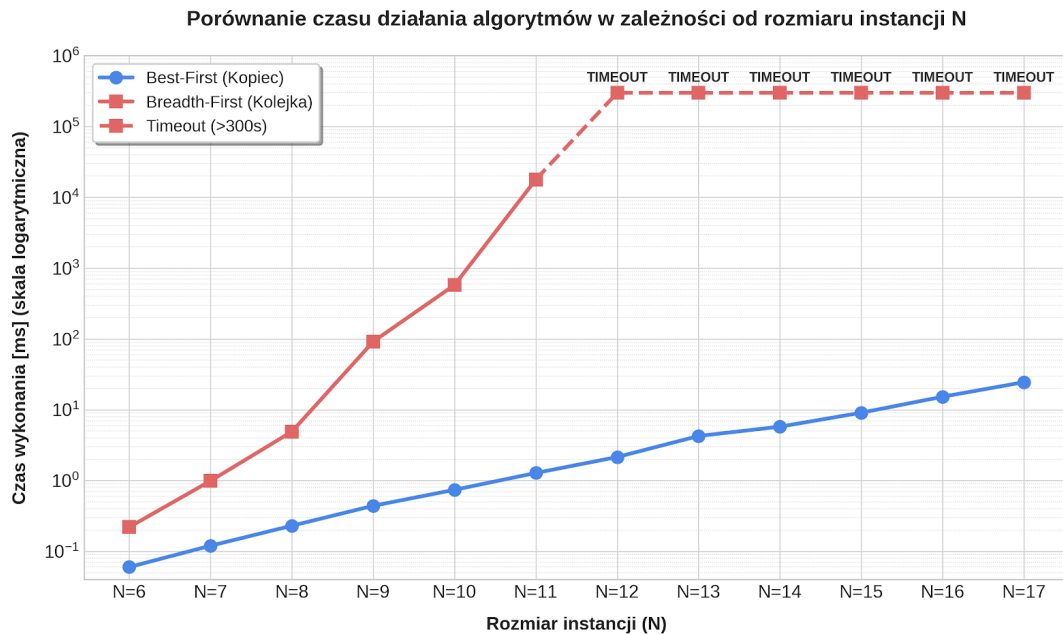
Jak można zauważyć, dla rozmiaru $N = 14$ czas przekraczał minutę działania. W przypadku metody Branch and Bound instancje tego rozmiaru rozwiązywane są w ułamku sekundy, co dowodzi potęgi mechanizmu "ucinania" gałęzi.

5.2 Analiza czasów dla Branch and Bound

W poniższej tabeli i na wykresie zestawiono uśrednione czasy działania dwóch wariantów algorytmu Branch and Bound.

Tabela 3: Uśredniony czas działania wariantów algorytmu B&B (100 prób na każde N)

N	Czas B&B Kopiec [ms]	Czas B&B Kolejka [ms]	% przerwanych (Kolejka)
6	0,06	0,22	0%
7	0,12	0,99	0%
8	0,23	4,91	0%
9	0,44	91,85	0%
10	0,74	579,66	0%
11	1,28	17931,10	3%
12	2,15	> 300 000	100%
13	4,26	> 300 000	100%
14	5,77	> 300 000	100%
15	9,08	> 300 000	100%
16	15,28	> 300 000	100%
17	24,53	> 300 000	100%



Rysunek 1: Porównanie czasu działania algorytmów w zależności od rozmiaru instancji N

5.3 Wnioski końcowe

Implementacja algorytmu metodą Podziału i Ograniczeń (Branch and Bound) pozwoliła na drastyczne skrócenie czasu wyznaczania optymalnego cyklu Hamiltona w porównaniu do klasycznego Przeglądu Zupełnego (Brute-Force).

Zestawiając ze sobą strategie przeszukiwania drzewa rozwiązań, można zauważyć, że metoda wykorzystująca Kopiec Min (*Best-First Search*) wykazuje przewagę wydajnościową przy większych instancjach problemu nad metodą wykorzystującą zwykłą kolejkę FIFO (*Breadth-First Search*). Jest to spowodowane mechanizmem triazu – kopiec na każdym etapie promuje najbardziej perspektywiczne gałęzie (o najmniejszym szacowanym `lowerBound`). Dzięki temu algorytm szybciej dociera do "dobrego" rozwiązania całkowitego, co w konsekwencji pozwala na znacznie wcześniejsze obcinanie pozostałych, mniej obiecujących ścieżek z pamięci operacyjnej.

Literatura

- [1] Cormen T. H., Leiserson C. E., Rivest R. L., Stein C., *Wprowadzenie do algorytmów*, Wydawnictwo Naukowe PWN, Warszawa 2012.
- [2] Wróblewski P., *Algorytmy, struktury danych i techniki programowania*, Wydawnictwo Helion, Gliwice 2015.
- [3] Papadimitriou C. H., *Złożoność obliczeniowa*, Wydawnictwa Naukowo-Techniczne, Warszawa 2002.
- [4] @avnishcodes, *Działanie kolejki fifo*, <https://www.youtube.com/@avnishcodes/shorts>, Youtube.